NAME :- KISHORI KAMALI

REG NO: 23MIC0055

SUBJECT:- ADVANCED PYTHON PROGRAMMING

QUESTION 1:-

Link:- https://leetcode.com/problems/longest-increasing-path-in-a-matrix/

Longest Increasing Path in a Matrix

Reframed question:
Create a MatrixPathFinder class that stores an integer matrix and provides a method findLongestIncreasingPath() to find the length of the longest path in which each next cell contains a greater value. Movement is allowed only up, down, left, or right. The class should internally manage matrix boundaries and path calculations.

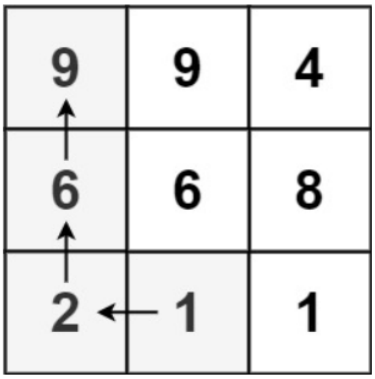
Hint for faculty:

- Create a class MatrixPathFinder.
- Store matrix, rows, and columns as attributes.
- Use DFS + memoization for each cell.
- Create a helper method such as dfs(row, col).

Why class?
The matrix and its dimensions belong naturally to one object.
The class encapsulates both the data and path-finding operations.
Helper methods can access the matrix without repeatedly passing it as an argument.
This makes the solution reusable for different matrices.

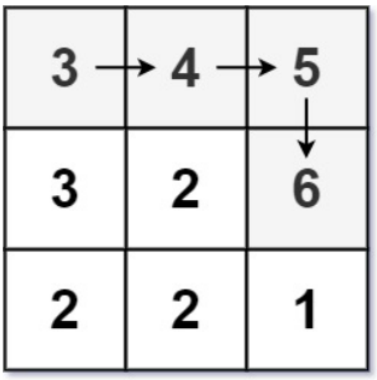
Excepted Input and Output:-

Example 1:




```
Input: matrix = [[9,9,4],[6,6,8],[2,1,1]]
Output: 4
Explanation: The longest increasing path is [1, 2, 6, 9].
```

Example 2:




```
Input: matrix = [[3,4,5],[3,2,6],[2,2,1]]
Output: 4
Explanation: The longest increasing path is [3, 4, 5, 6]. Moving diagonally is not allowed.
```

Example 3:

```
Input: matrix = [[1]]
Output: 1
```

QUESTION 2:-

Link:- https://leetcode.com/problems/sliding-window-maximum/

Sliding Window Maximum

Reframed question:
Create a SlidingWindowAnalyzer class that accepts an integer array and a window size k. Implement a method findMaximums() that returns the maximum value from every sliding window as the window moves from left to right.

Hint for faculty:

- Create class SlidingWindowAnalyzer.
- Store nums and k.
- Use a **deque** to maintain indices of useful elements.
- Remove indices that are outside the current window.
- Remove smaller elements from the back of the deque.

Why class?
The array and window size form the state of a single sliding-window analysis.
The class keeps this state together and hides the deque-based implementation.
The same object can perform the analysis without passing nums and k repeatedly.
This improves organization and makes the algorithm reusable.

Excepted Input and Output:-

Example 1:

```
Input: nums = [1,3,-1,-3,5,3,6,7], k = 3
Output: [3,3,5,5,6,7]
Explanation:
Window position                Max
---------------                -----
[1  3  -1] -3  5  3  6  7       3
 1 [3  -1  -3] 5  3  6  7       3
 1  3 [-1  -3  5] 3  6  7       5
 1  3  -1 [-3  5  3] 6  7       5
 1  3  -1  -3 [5  3  6] 7       6
 1  3  -1  -3  5 [3  6  7]      7
```

Example 2:

```
Input: nums = [1], k = 1
Output: [1]
```

QUESTION 3:-

Link:- https://leetcode.com/problems/continuous-subarrays/

Continuous Subarrays

Reframed question:
Create a ContinuousSubarrayAnalyzer class that stores an integer array and provides a method countContinuousSubarrays(). A subarray is considered continuous when the difference between its maximum and minimum values is at most 2. Return the total number of continuous subarrays.

Hint for faculty:

- Create class ContinuousSubarrayAnalyzer.
- Store nums as an attribute.
- Maintain a sliding window.
- Keep track of the current minimum and maximum using two monotonic deques.
- If max - min > 2, move the left boundary forward.

Why class?
The array and the current-window analysis belong to one logical object.

The class hides the complexity of maintaining minimum and maximum values.
The same analyzer can be reused with different input arrays.
This provides better encapsulation than placing all logic in one standalone function.

Excepted Input and Output:-

Example 1:

```
Input: nums = [5,4,2,4]
Output: 8
Explanation:
Continuous subarray of size 1: [5], [4], [2], [4].
Continuous subarray of size 2: [5,4], [4,2], [2,4].
Continuous subarray of size 3: [4,2,4].
There are no subarrys of size 4.
Total continuous subarrays = 4 + 3 + 1 = 8.
It can be shown that there are no more continuous subarrays.
```

Example 2:

```
Input: nums = [1,2,3]
Output: 6
Explanation:
Continuous subarray of size 1: [1], [2], [3].
Continuous subarray of size 2: [1,2], [2,3].
Continuous subarray of size 3: [1,2,3].
Total continuous subarrays = 3 + 2 + 1 = 6.
```

QUESTION 4:-

Link:- https://leetcode.com/problems/flatten-binary-tree-to-linked-list/

Flatten Binary Tree to Linked List

This is actually the **strongest question for demonstrating OOP/classes**, because the problem already uses a TreeNode class.

Reframed question:
Create a BinaryTreeFlattener class that accepts the root of a binary tree and provides a method flatten() to convert the tree into a linked-list structure in preorder. The original TreeNode objects must be reused, with left = null and right pointing to the next node.

Hint for faculty:

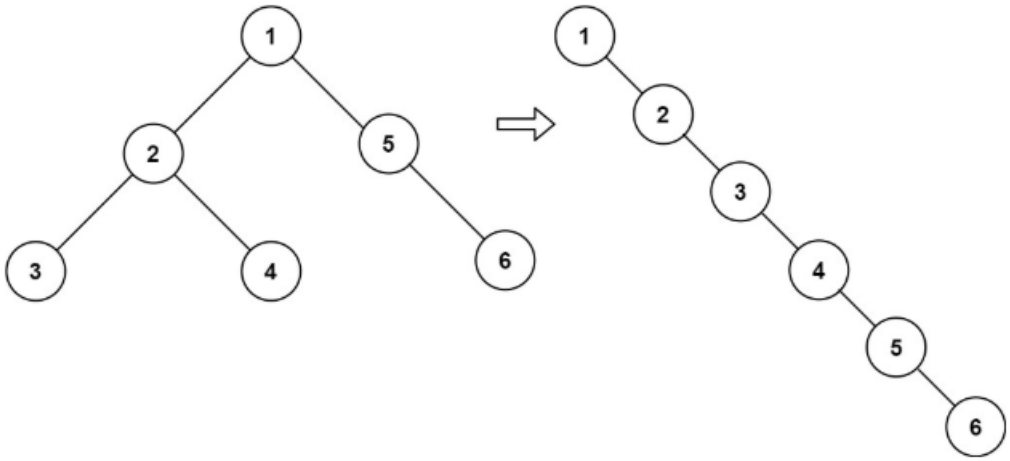
- Create/use a TreeNode class with val, left, and right.
- Create a BinaryTreeFlattener class.
- Implement flatten(root).
- Think about preorder traversal: root → left → right.
- The tree can be rearranged in-place without creating a new linked list.

Why class?
A binary tree is naturally represented as interconnected TreeNode objects.
The flattener class encapsulates the tree transformation operation.

It allows the tree structure and its transformation logic to remain organized.
Using objects also directly represents the parent-child relationships of the tree.

Excepted Input and Output:-

Example 1:




```
Input: root = [1,2,5,3,4,null,6]
Output: [1,null,2,null,3,null,4,null,5,null,6]
```

Example 2:

```
Input: root = []
Output: []
```

Example 3:

```
Input: root = [0]
Output: [0]
```

QUESTION 5:-

Link:- https://leetcode.com/problems/delete-greatest-value-in-each-row/

Delete Greatest Value in Each Row

Reframed question:
Create a GridValueProcessor class that stores a 2D integer grid and provides a method calculateScore(). In each round, remove the largest element from every row, find the largest among the removed elements, and add it to the total score. Continue until all elements are removed.

Hint for faculty:

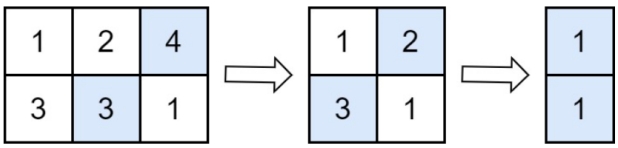
- Create class GridValueProcessor.

- Store grid.
- Sort every row first.
- Repeatedly take the last element from each row.
- Find the maximum among those values and add it to the answer.

Why class?
The grid and its processing operations are closely related data and behavior.
The class encapsulates the grid manipulation and score calculation.
It makes the operation reusable for different grids.
This is cleaner than keeping the grid and processing logic as unrelated global data/functions.

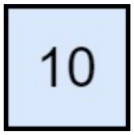
Excepted Input and Output:-

Example 1:




```
Input: grid = [[1,2,4],[3,3,1]]
Output: 8
Explanation: The diagram above shows the removed values in each step.
- In the first operation, we remove 4 from the first row and 3 from the second row (notice that, there are two cells with value 3 and we
can remove any of them). We add 4 to the answer.
- In the second operation, we remove 2 from the first row and 3 from the second row. We add 3 to the answer.
- In the third operation, we remove 1 from the first row and 1 from the second row. We add 1 to the answer.
The final answer = 4 + 3 + 1 = 8.
```

Example 2:




```
Input: grid = [[10]]
Output: 10
Explanation: The diagram above shows the removed values in each step.
- In the first operation, we remove 10 from the first row. We add 10 to the answer.
The final answer = 10.
```

QUESTION 6:-

Link:- https://leetcode.com/problems/find-if-path-exists-in-graph/

Find if Path Exists in Graph

Reframed question:
Create a GraphPathFinder class that represents a bidirectional graph using n vertices and a list of edges. Implement a method hasPath(source, destination) that determines whether a path exists between the given vertices.

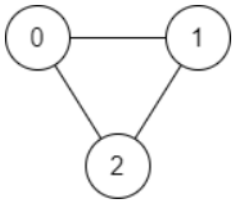
Hint for faculty:

- Create class GraphPathFinder.
- Store n and the graph's adjacency list.
- Build the adjacency list in the constructor.
- Use BFS or DFS to search from source.
- Maintain a visited array/set.

Why class?
A graph consists of persistent data such as vertices and edges.
The class can encapsulate the graph representation and traversal operations.
The same graph object can answer multiple path queries without rebuilding the graph.
This demonstrates a clear separation between graph data and graph-processing behavior.

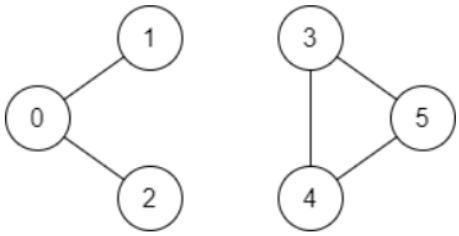
Expected Input and Output:-

Example 1:




```
Input: n = 3, edges = [[0,1],[1,2],[2,0]], source = 0, destination = 2
Output: true
Explanation: There are two paths from vertex 0 to vertex 2:
- 0 → 1 → 2
- 0 → 2
```

Example 2:




```
Input: n = 6, edges = [[0,1],[0,2],[3,5],[5,4],[4,3]], source = 0, destination = 5
Output: false
Explanation: There is no path from vertex 0 to vertex 5.
```